

Task 1 - Web App QA & Debug Report

Automation & QA Developer take-home assessment

Application under test: Conduit (RealWorld Angular demo) at <https://demo.realworld.show> | API: <https://api.realworld.show>

Author: Nishant Sinha | Date: 29 September 2026 | Time-boxed to about 75 minutes

1. Scope and method

The brief lists <https://demo.realworld.io> as the public demo. That address now returns a 404, so I used the current official demo at <https://demo.realworld.show> (Angular front end, API at api.realworld.show). I walked the main flows in a real browser: sign-up, login, create, edit, delete and logout, plus favourite and follow as a guest and as a signed-in user. I used DevTools (Network, Application, throttling) and, for backend behaviour, direct API calls with curl, so front-end bugs could be told apart from API bugs. Assumption: the demo's per-session data isolation is intentional, so I did not file it as a defect.

2. What works well

- Logging out and pressing Back returns to the sign-in page with empty fields and no logged-in content visible.
- On a throttled Slow 3G connection the home feed shows a "Loading articles..." state instead of a blank page.
- Reading the global feed works, and the API correctly answers 401 when a guest tries a protected action.

3. Bug table

Severity uses the scale from the brief. Nothing stops happy-path reading, so nothing is Critical, but the High item and several Mediums would block a production launch. Ordered by severity.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Sign-up accepts invalid email, 1-character password and duplicate accounts	1. Open Sign up. 2. Enter username a , email q (or not-an-email), password x . 3. Submit and watch DevTools > Network. 4. Repeat with the same email and username (or POST <code>/api/users</code> with curl).	Expected: Malformed email and short password are rejected; duplicates are refused with a 422. Actual: POST <code>/api/users</code> returns 201 and a session token for all of these, including the repeat registration.	High	Suspected: the API has no format, length or uniqueness validation. The front end adds none either: the email input is <code>type=text</code> and the form has <code>novalidate</code> .
2	Missing or deleted article, profile or editor target shows a blank page or empty form	1. While logged in, open <code>/article/does-not-exist</code> and <code>/profile/no-such-user</code> . 2. Delete one of your own articles, then press Back or reopen its URL. 3. Open the editor URL for that deleted article.	Expected: A clear "not found" message with a link back to Home; the editor does not open for a deleted article. Actual: Article and profile pages show only the header and footer, with no error or retry. The editor opens an empty form.	Medium	Suspected: 404 responses from the API are not handled, and the page templates render nothing when data is missing.
3	Username with special characters is accepted, then the profile page fails	1. Sign up with username ni[o . 2. Finish sign-up and open your profile. 3. Check DevTools > Network.	Expected: Usernames that cannot work in a URL are rejected at sign-up, or the profile loads normally. Actual: The account is created, then the profile request returns 404.	Medium	Suspected: no character rules on usernames, and the raw value is used in the profile route or request URL.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
4	No confirmation before deleting an article	1. Open one of your own articles. 2. Click Delete Article (it sits right next to Edit Article).	Expected: A confirmation step or an undo for an irreversible action. Actual: The article is removed immediately with no undo. A misclick next to Edit destroys the content.	Medium	Suspected: the delete button calls the API directly with no confirm step in the UI.
5	Actions that need an account look usable when logged out	1. Log out. 2. Open any article. 3. Click Favorite Article or Follow on the author.	Expected: Buttons hidden or labelled "Sign in to favorite"; a click goes to Sign in and returns afterwards. Actual: Buttons look active. Favorite sends the user to /register with no explanation. The API itself correctly answers 401.	Medium	Suspected: the front end does not check login state before showing these buttons, and its 401 handling redirects to register instead of login.
6	Forms give no feedback: silent disabled buttons, no labels, wrong input types	1. Open Sign up and Sign in with empty fields. 2. Note the faded, disabled submit button and whether any message explains it. 3. Inspect the inputs in DevTools > Elements.	Expected: Visible messages saying what is missing; every input has a label; email uses type=email; autocomplete is set. Actual: The button is disabled with no explanation. Sign up has no <label> elements. Email is type=text, autocomplete is empty and the form has novalidate.	Medium	Suspected: placeholders are used as labels, and validation relies on disabling the button instead of showing messages.
7	CORS allows any origin together with credentials	Send OPTIONS https://api.realworld.show/api/users/login with headers Origin: https://evil.example and Access-Control-Request-Method: POST (curl).	Expected: Allow-Origin limited to a fixed list containing only the demo front end. Actual: The response echoes Access-Control-Allow-Origin: https://evil.example and adds Access-Control-Allow-Credentials: true. Practical impact here is limited because the token travels in a header, not a cookie.	Medium	Suspected: the CORS middleware reflects the request Origin instead of checking it against an allow-list.
8	Grey secondary text may be hard to read (possible contrast concern)	1. Open the home feed. 2. Look at the lighter grey text, such as "Read more..." and article dates, against the white background. 3. Compare it with the main body text.	Expected: Secondary text is as easy to read as the body text, in line with accessibility guidance on colour contrast (WCAG 1.4.3). Actual: The grey text looks noticeably lighter than the body text. I did not measure an exact contrast ratio, so this is a potential accessibility concern, not a confirmed WCAG failure.	Low	Suspected: the shared theme uses a light grey for secondary text. A contrast checker would confirm whether it meets the guideline.
9	Login token kept in localStorage	1. Log in. 2. Open DevTools > Application > Local Storage. 3. Find the stored auth token.	Expected: Token held in an httpOnly, Secure cookie, or kept short-lived. Actual: The token is readable by any script running on the page, so a future XSS bug would expose every session.	Low	Suspected: the simplest persistence pattern was used. The risk is theoretical today because HTML in article content is sanitised on display.

4. Root-cause analysis - Issue 1 (missing validation on sign-up)

When someone signs up with username "a", email "q" and password "x", the API accepts it, returns HTTP 201 and issues a session token. Registering the same email and username again also returns 201, so the API is not checking for duplicates either. The sign-up page does not compensate: the email box is a plain text field, the form has novalidate switched on, and the only guard is a submit button that stays disabled without saying why. The likely reason is that the API only checks that fields are present, not that their contents are valid; I could not see the backend code, so this is inferred from its responses. The result is that junk accounts can be created and one person can register the same identity repeatedly. On the server I would validate the email format, require a password of at least 8 characters, add unique indexes on email and username, and return HTTP 422 with the field errors the front end already expects. On the client I would switch the email input to type=email, add visible labels, and show the server's 422 messages under the form. I would also add one automated contract test: POST /api/users with a bad email must return 422, and a second POST with the same email must also return 422. That test would have caught this before any user did. I chose this

issue because it is the one a non-technical stakeholder feels most directly: anyone can join with garbage data.

5. Also observed, not filed

- Password can be changed without entering the current one. This matches the RealWorld spec, so I treat it as a hardening idea, not a defect.
- Session isolation and seed articles dated "today": documented demo behaviour.
- Every seeded author shares one default avatar: cosmetic.
- HEAD /api/articles returns 405: harmless for browsers, but an uptime monitor should use GET.
- HTML typed into article titles is stored by the API but sanitised on display, so I did not call it a live XSS.

6. Launch recommendation

Do not put this front end and API on a public domain as they are. Reading the feed works, but everything that creates an identity or changes data needs a hardening pass: validation, unique users, delete confirmation, 404 pages, login-aware buttons and a CORS allow-list. Each is a small change, so I would estimate days, not weeks.